

Отчет о практическом задании

Ансамблевые методы машинного обучения для задач регрессии: реализация веб-сервера

Практикум 317 группа, ММП ВМК МГУ

Сычев Владимир Александрович

Декабрь 2025

1 Введение

Данная работа посвящена исследованию ансамблевых методов машинного обучения для решения задачи регрессии, а также разработке программной системы, обеспечивающей их практическое применение. В рамках задания рассматриваются методы случайного леса и градиентного бустинга, реализованные самостоятельно на языке Python. В качестве базовых моделей ансамблей используются решающие деревья.

В ходе работы осуществляется экспериментальное исследование качества ансамблевых алгоритмов регрессии на датасете данных о продажах недвижимости House Sales in King County, USA. А также проводится анализ поведения алгоритмов в зависимости от параметров моделей.

Основные задачи работы:

- реализация методов случайного леса и градиентного бустинга для задачи регрессии с использованием стандартных средств языка Python и библиотеки `numpy`;
- проведение вычислительных экспериментов на выбранном датасете и анализ полученных результатов;
- разработка веб-сервера, предоставляющего доступ к реализованным моделям.

2 Обработка данных

2.1 Предобработка данных

Обучающая выборка содержит 21 613 объектов и 21 признак. Пример данных приведен в таблице 1.

	id	date	price	bedrooms	bathrooms	sqft_living	...
0	7129300520	20141013T000000	221900.0	3	1.00	1180	...
1	6414100192	20141209T000000	538000.0	3	2.25	2570	...
2	5631500400	20150225T000000	180000.0	2	1.00	770	...
3	2487200875	20141209T000000	604000.0	4	3.00	1960	...
4	1954400510	20150218T000000	510000.0	3	2.00	1680	...

Таблица 1: Фрагмент датасета с характеристиками недвижимости

Все признаки, за исключением `id` и `date`, являются информативными для решаемой задачи, поэтому столбцы `id` и `date` были удалены из датасета. Оставшиеся признаки являются вещественными. Пропусков в данных нет. Стандартизация признаков не применялась, поскольку в качестве базовых моделей ансамблей используются решающие деревья.

Данные были разделены на обучающую и контрольную выборки в пропорции 4:1. Перед разделением выборка была перемешана случайным образом¹.

3 Исследование поведения алгоритма случайного леса для задачи регрессии

Будем изучать зависимость RMSE (Root Mean Square Error) и времени работы алгоритма от следующих факторов: количество деревьев в ансамбле (`n_estimators`), размерность подвыборки признаков для одной вершины дерева (`max_features`) и максимальная глубина дерева (`max_depth`).

3.1 Анализ `max_features` и `max_depth`

Чтобы избежать тройного перебора гиперпараметров, зафиксируем значение `n_estimators = 50` и будем перебирать только `max_features` и `max_depth`.

¹Для случайного перемешивания был использован генератор псевдослучайных чисел с фиксированным значением `seed = 52`.

Рассматриваемые диапазоны гиперпараметров:

`max_features = [1, 2, 3, 4, 5, 6, 9, 12, 15, 18]`, `max_depth = [3, 5, 8, 12, 20, None2]`

На рисунках 1 и 2 представлены зависимости RMSE и времени от гиперпараметров `max_features` и `max_depth`.

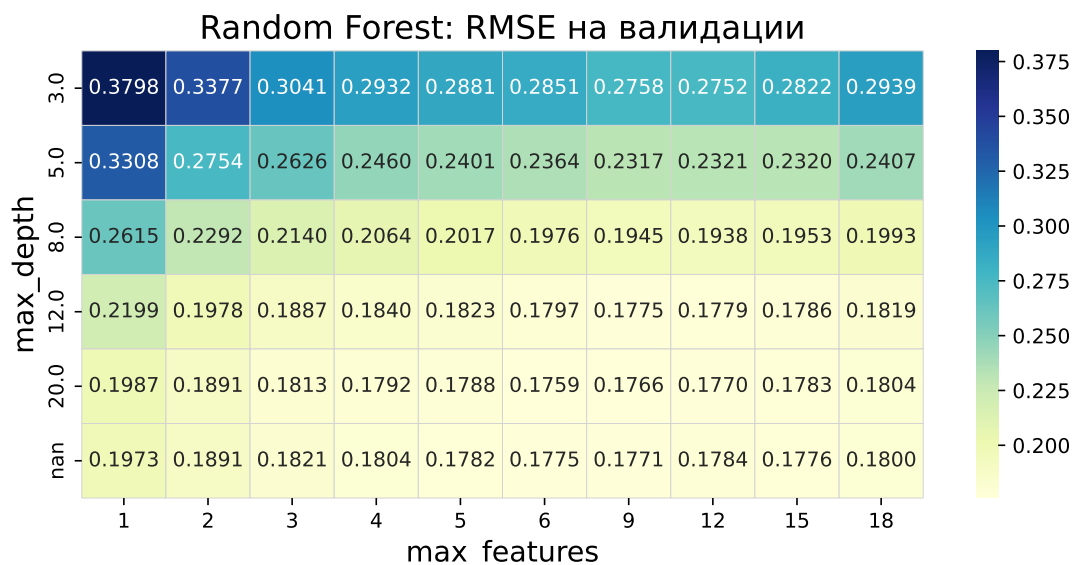


Рис. 1: Тепловая карта, показывающая зависимость RMSE от гиперпараметров `max_features` и `max_depth`

На тепловой карте 1, отображающей RMSE на валидации, видно, что ошибка модели существенно зависит от глубины деревьев и числа признаков, используемых при разбиении. При небольшой глубине дерева, равной 3, RMSE достаточно высока и варьируется от 0.2752 до 0.3798 в зависимости от числа признаков. При глубине 5 минимальное значение RMSE снижается до 0.2317, а при глубине 8 — до 0.1938. Увеличение глубины до 12 — 20 заметно снижает ошибку, достигая минимальных значений 0.1759 — 0.1770 при `max_depth = 20` и `max_features` в диапазоне 6 — 12. Неограниченная глубина не улучшает точность. В некоторых случаях наблюдается рост ошибки, что может быть связано с переобучением.

Зависимость RMSE от числа признаков показывает, что увеличение `max_features` сначала снижает ошибку, однако после определенного значения улучшение прекращается. Например, при `max_depth = 12` RMSE снижается от 0.2199 до 0.1775 при `max_features` от 1 до 9, а после RMSE возрастает до 0.1819.

Анализ тепловой карты времени обучения 2 показывает, что время растет с увеличением глубины деревьев и числа признаков. При глубине 3 модель обучается менее чем за 2 секунды, тогда как при глубине 20 время обучения достигает 8 секунд.

²Значение `None` для гиперпараметра `max_depth` соответствует неограниченной глубине дерева.

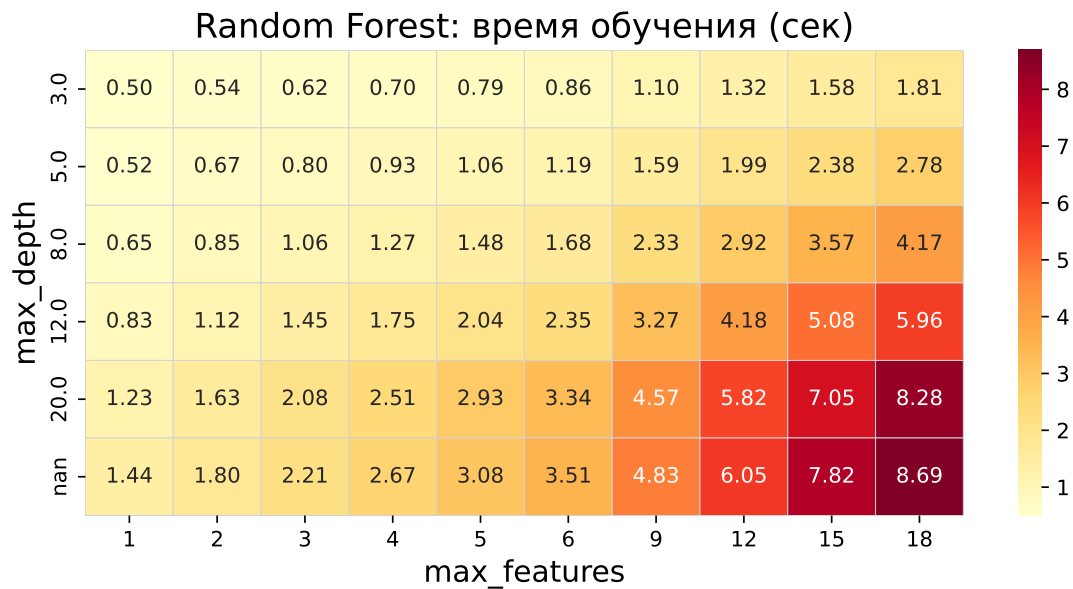


Рис. 2: Тепловая карта, показывающая зависимость времени работы алгоритма от гиперпараметров `max_features` и `max_depth`

Оптимальным сочетанием с точки зрения баланса качества и скорости является глубина дерева 20 и число признаков 6. Эти параметры обеспечивают минимальную RMSE на валидации при умеренном времени обучения, что делает их предпочтительным выбором для настройки модели случайного леса в данной задаче.

3.2 Анализ `n_estimators`

Теперь зафиксируем `max_features = 20` и `max_depth = 6` и будем варьировать количество деревьев в ансамбле: `n_estimators = [5, 10, 20, 50, 100, 200, 500, 1000]`.

На рисунке 3 представлена зависимость RMSE от количества деревьев в ансамбле. Анализ результатов показывает, что увеличение числа деревьев ведет к снижению ошибки. Это объясняется уменьшением дисперсии предсказаний за счет усреднения большего числа независимых моделей.

Из графика видно, что существенное улучшение качества наблюдается при увеличении числа деревьев от 5 до 100 (RMSE снижается от 0.2008 до 0.1764). После 100 деревьев ошибка продолжает снижаться, но с меньшей скоростью: при 200 деревьях RMSE составляет 0.1762, при 500 — 0.1754, а при 1000 деревьях достигает значения 0.1749.

Также можно отметить, что время обучения модели демонстрирует близкую к линейной зависимость от числа деревьев: обучение 100 деревьев занимает 6.76 с, 200 деревьев — 13.67 с, 500 деревьев — 34.02 с. Такое поведение ожидаемо, поскольку деревья в случайном лесу обучаются независимо.

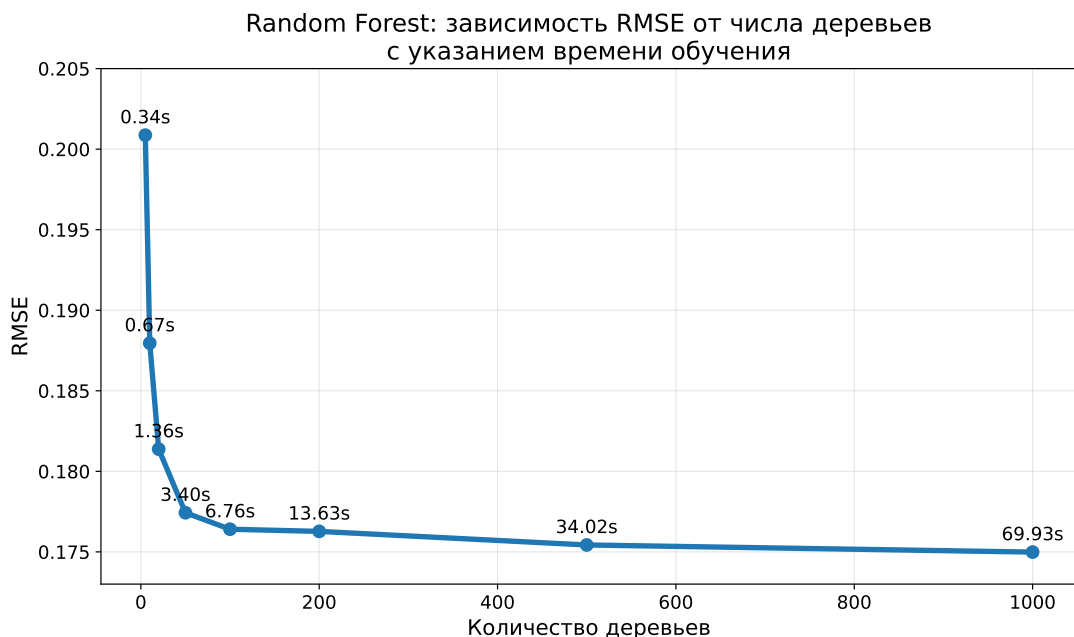


Рис. 3: Зависимость RMSE на валидации от числа деревьев в ансамбле

Таким образом, выбор `n_estimators` представляет собой компромисс между качеством предсказания и вычислительными затратами. К примеру, в данной задаче можно взять 200 — 500 деревьев, что обеспечивает хорошее качество при сравнительно небольшом времени обучения.

4 Исследование поведения алгоритма градиентного бустинга для задачи регрессии

Будем изучать зависимость RMSE и времени работы алгоритма от следующих факторов: количество деревьев в ансамбле (`n_estimators`), размерность подвыборки признаков для одной вершины дерева (`max_features`), максимальная глубина дерева (`max_depth`) и темп обучения (`learning_rate`).

4.1 Анализ `max_features` и `max_depth`

Аналогично разделу 3.1 зафиксируем значения `n_estimators = 50` и `learning_rate = 0.01`. Будем перебирать только `max_features` и `max_depth`.

Рассматриваемые диапазоны гиперпараметров:

`max_features = [1, 2, 3, 4, 5, 6, 9, 12, 15, 18]`, `max_depth = [3, 5, 8, 12, 20, None]`

На рисунках 4 и 5 представлены зависимости RMSE и времени от гиперпараметров `max_features` и `max_depth`.

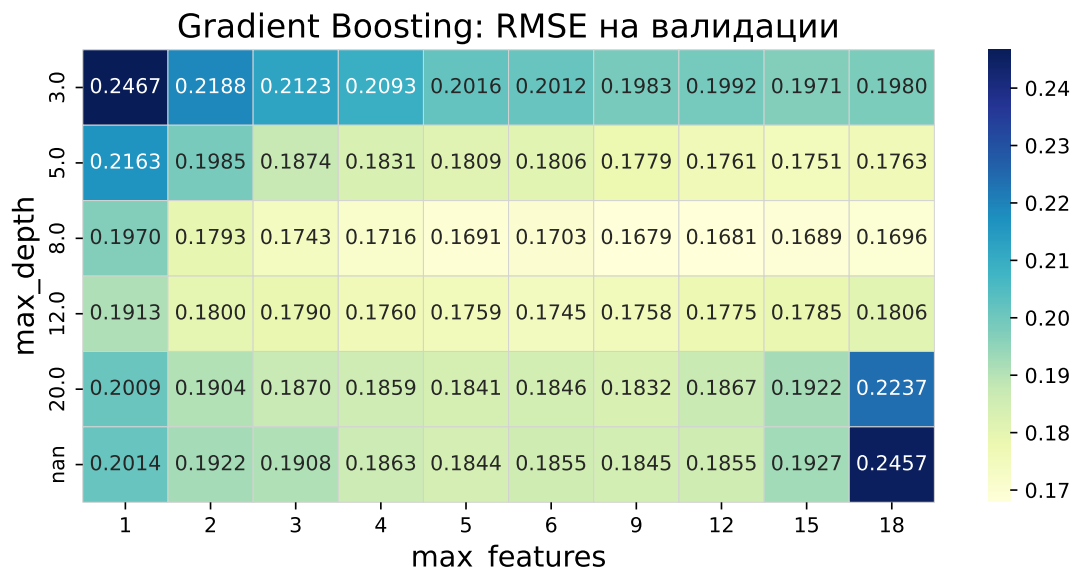


Рис. 4: Тепловая карта, показывающая зависимость RMSE от гиперпараметров `max_features` и `max_depth`

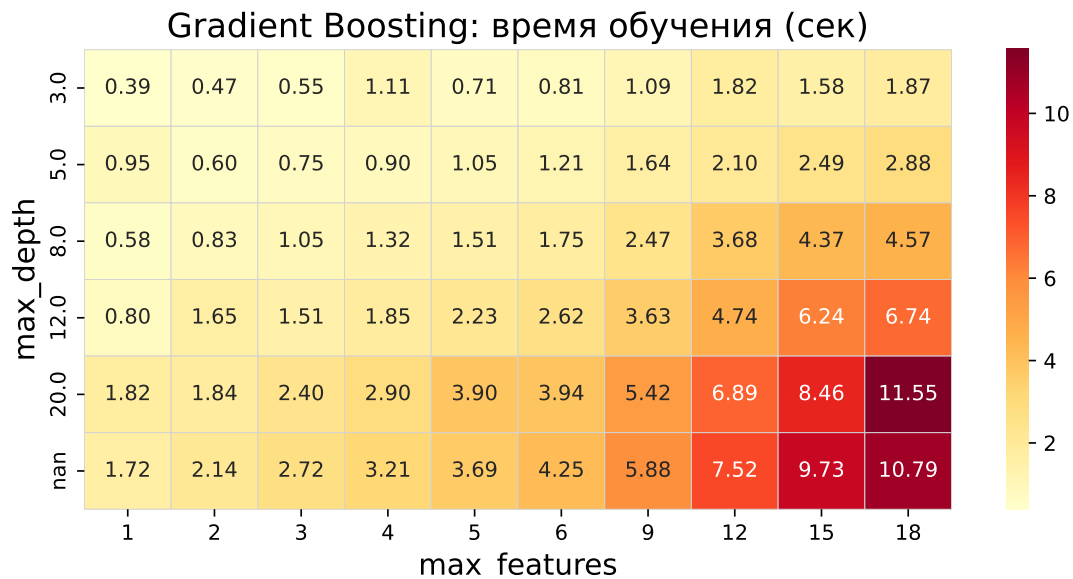


Рис. 5: Тепловая карта, показывающая зависимость времени работы алгоритма от гиперпараметров `max_features` и `max_depth`

На тепловой карте 4, отображающей RMSE на валидации, видно, что ошибка модели существенно зависит от глубины деревьев и числа признаков, используемых при разбиении. При небольшой глубине дерева, равной 3, RMSE достаточно

высока и варьируется от 0.1971 до 0.2467 в зависимости от числа признаков. При глубине 5 минимальное значение RMSE снижается до 0.1751. Увеличение глубины до 8 заметно снижает ошибку, достигая минимальных значений 0.1679 — 0.1703 при `max_features` в диапазоне 5 — 12. Дальнейшее увеличение глубины только уменьшает точность: при неограниченной глубине и глубине 20 RMSE доходит до 0.2237 — 0.2457

Зависимость RMSE от числа признаков показывает, что увеличение `max_features` сначала снижает ошибку, однако после определенного значения улучшение прекращается. Например, при `max_depth = 12` RMSE снижается от 0.1913 до 0.1745 при `max_features` от 1 до 6, после RMSE начинает расти до 0.1806.

Анализ тепловой карты времени обучения 5 показывает, что время растет с увеличением глубины деревьев и числа признаков. При глубине 3 модель обучается менее чем за 2 секунды, тогда как при глубине 20 время обучения достигает 11 секунд.

Оптимальным сочетанием с точки зрения баланса качества и скорости является глубина дерева 8 и число признаков 9. Для сравнения, оптимальные параметры для случайного леса (раздел 3.1) были: глубина дерева 20 и число признаков 6. То есть градиентный бустинг в данной задаче требует меньшей глубины деревьев (8 против 20), но большего числа признаков при разбиении (9 против 6).

4.2 Анализ `n_estimators`

Теперь зафиксируем `max_features = 8`, `max_depth = 9` и `learning_rate = 0.01` и будем варьировать количество деревьев в ансамбле: `n_estimators = [5, 10, 20, 50, 100, 200, 500, 1000]`.

На рисунке 6 представлена зависимость RMSE от количества деревьев в ансамбле. Анализ результатов показывает, что увеличение числа деревьев ведет к снижению ошибки. Это объясняется уменьшением дисперсии предсказаний за счет усреднения большего числа независимых моделей.

Из графика видно, что существенное улучшение качества модели наблюдается при увеличении числа деревьев от 5 до 100 (RMSE снижается от 0.3554 до 0.1669). После 100 деревьев ошибка начинает расти: при 200 деревьях RMSE составляет 0.1672, при 500 — 0.1693, а при 1000 деревьях достигает значения 0.1723. Для сравнения, после 100 деревьев RMSE случайного леса (раздел 3.1) продолжала уменьшаться.

Также можно отметить, что время обучения модели демонстрирует близкую к линейной зависимость от числа деревьев: обучение 100 деревьев занимает 5.97 с, 200 деревьев — 12.35 с, 500 деревьев — 30.23 с. Однако, в отличие от случайного леса, деревья в градиентном бустинге обучаются последовательно, поскольку каждое последующее дерево строится для аппроксимации ошибок предыдущих

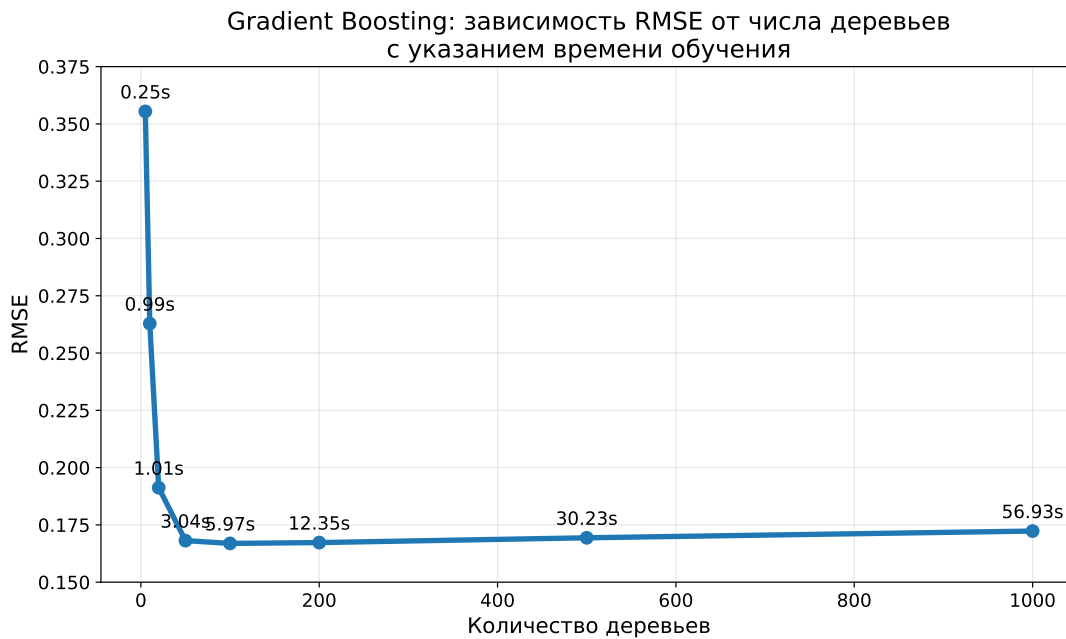


Рис. 6: Зависимость RMSE на валидации от числа деревьев в ансамбле

моделей. Общее время обучения остается приблизительно линейным относительно числа деревьев, что объясняется относительно постоянной сложностью построения одного дерева на каждом шаге алгоритма.

Таким образом, оптимальное значение `n_estimators` = 100: до 100 деревьев в ансамбле RMSE уменьшается, после 100 деревьев RMSE увеличивается.

4.3 Анализ `learning_rate`

Теперь зафиксируем `max_features` = 8, `max_depth` = 9 и `n_estimators` = 100 и будем варьировать темп обучения в ансамбле: `learning_rate` = [0.01, 0.03, 0.05, 0.1, 0.15, 0.2, 0.3].

На рисунке 7 представлена зависимость RMSE от темпа обучения.

Из графика видно, что улучшение качества наблюдается при увеличении темпа обучения от 0.01 до 0.1 (RMSE снижается от 0.2661 до 0.1664). Такой выигрыш объясняется тем, что при слишком малом темпе обучения (0.01) модель сходится медленно и не достигает оптимума за фиксированное число итераций.

После оптимального значения 0.1 ошибка начинает монотонно расти: при темпе обучения 0.15 RMSE составляет 0.1689, при 0.2 — 0.1705, а при 0.3 достигает значения 0.1778. Слишком большой темп обучения приводит к «перешагиванию» оптимума на каждой итерации, что ухудшает сходимость алгоритма и увеличивает итоговую ошибку.

Интересно отметить, что время обучения модели при различных значениях `learning_rate` остается практически неизменным и составляет около 5 секунд.

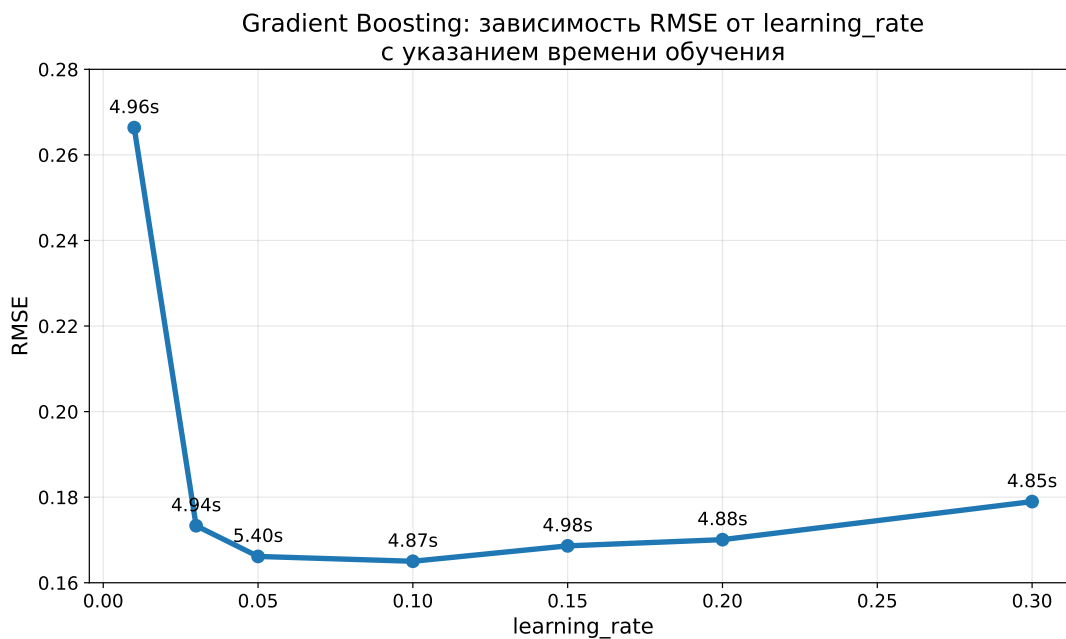


Рис. 7: Зависимость RMSE на валидации от темпа обучения

Это объясняется тем, что данный параметр не влияет на вычислительную сложность построения отдельных деревьев.

Таким образом, оптимальное значение `learning_rate` для данной задачи является 0.1.

5 Веб-сервер

5.1 Общая идея и назначение

В рамках данной работы был реализован веб-сервер, предоставляющий HTTP API для взаимодействия с моделью машинного обучения. Основная цель интерфейса — дать возможность пользователю без знания языка Python выполнять полный цикл работы с моделью:

1. создавать и настраивать эксперименты;
2. загружать датасеты;
3. запускать обучение;
4. анализировать процесс обучения;
5. выполнять инференс.

Веб-сервер реализован с использованием фреймворка FastAPI и предоставляет REST API для работы ML-моделей. Клиентская часть выполнена на Streamlit, специализированном фреймворке для создания веб-интерфейсов Data Science приложений. Streamlit-интерфейс отправляет HTTP-запросы к FastAPI серверу и визуализирует полученные результаты.

5.2 Архитектура решения

Решение состоит из двух логически независимых сервисов:

1. Backend (FastAPI). Отвечает за хранение информации об экспериментах, загрузку и обработку данных, обучение модели, сохранение состояния модели, выполнение предсказаний.
2. Frontend (Streamlit). Отвечает за пользовательский интерфейс, отправку HTTP-запросов к backend.

Взаимодействие между сервисами осуществляется исключительно через HTTP API.

Для обеспечения удобного запуска и переносимости решение обернуто в Docker. Используются: отдельный Docker-образ для backend, отдельный Docker-образ для frontend, файл docker-compose.yml, который поднимает оба сервиса одновременно.

Такой подход позволяет запускать приложение одной командой и разворачивать его на любом устройстве, поддерживающем Docker, независимо от особенностей локального окружения и установленных зависимостей.

6 Заключение

В ходе работы были исследованы ансамблевые методы машинного обучения — случайный лес и градиентный бустинг — на задаче предсказания цен недвижимости с использованием решающих деревьев в качестве базовых моделей.

Проведен комплексный анализ влияния гиперпараметров на качество моделей, включая глубину деревьев, количество признаков при разбиении, число деревьев в ансамбле и темп обучения для градиентного бустинга.

Случайный лес показал лучшие результаты при глубине деревьев 20 и 6 признаках на разбиении, достигая RMSE 0.1749. Градиентный бустинг продемонстрировал более высокую точность (RMSE 0.1664) при меньшей глубине (8) и большем числе признаков (9).

Независимое обучение деревьев в случайном лесу требует большей индивидуальной сложности, тогда как градиентный бустинг компенсирует меньшую глубину деревьев их последовательной оптимизацией.

Разработана веб-система на FastAPI и Streamlit с Docker-контейнеризацией предоставляющая удобный интерфейс для практического использования моделей, абстрагируя пользователя от технических деталей реализации.